

Experiment: 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview:

The name "blockchain" comes from its structure data is organized in blocks, with each new block linked to the one before it, forming a continuous chain.

Each block contains crucial data, such as a list of transactions, a timestamp, and a unique identifier called a cryptographic hash. This hash is generated from the block's contents and the hash of the previous block, ensuring that each block is tightly connected to the one before it.

- Blockchain's linked structure makes data tampering detectable by altering hashes and breaking the chain.
- It acts as a distributed database, storing transactions across the network.
- Each transaction is verified by the majority, ensuring legitimacy.
- This decentralization prevents any single party from manipulating the data.

2. Mining in Blockchain

Mining is the process of validating and adding new transactions to the blockchain.

Miners compete to solve complex mathematical puzzles using computational power.

Purposes of mining:

- Confirms transactions
- Secures the network
- Prevents double spending

3. Multinode Blockchain Network

A multinode blockchain network consists of multiple interconnected nodes, where each node maintains a copy of the blockchain ledger.

Types of nodes:

- Full nodes: Store the complete blockchain and validate transactions.
- Light nodes: Store partial data and rely on full nodes.
- Mining nodes: Validate transactions and create new blocks.

Advantages of multinode networks:

- Fault tolerance: No single point of failure
- Decentralization: Increases trust and fairness
- Data consistency: All nodes synchronize using consensus mechanisms

If one node fails, the network continues to operate smoothly.

4. Consensus Mechanism

A consensus mechanism is a protocol that allows all nodes in a blockchain network to agree on the validity of transactions and the state of the ledger. Consensus ensures that all honest nodes reach the same blockchain state. They Prevent double spending, Maintain consistency across nodes, Ensure security and trust

Common consensus mechanisms:

- Proof of Work (PoW): Miners solve cryptographic puzzles (Bitcoin).
- Proof of Stake (PoS): Validators are chosen based on stake (Ethereum).

5. Transactions and Mining Reward

Transactions- A blockchain transaction represents the transfer of value or data between users.

Each transaction includes:

- Sender address

- Receiver address
- Amount
- Digital signature

Mining Reward- Miners receive rewards for successfully adding blocks:

1. Block reward: Newly created cryptocurrency.
2. Transaction fees: Fees paid by users.

Over time, block rewards may decrease (e.g., Bitcoin halving).

6. Chain Replacement

Chain replacement is a process used to resolve conflicts when multiple versions of the blockchain exist.

Sometimes, two miners may produce blocks at the same time, creating a temporary fork.

To handle this, blockchains follow the Longest Chain Rule:

- The chain with the most cumulative work or highest stake is considered valid.
- Nodes discard shorter or invalid chains.
- Transactions from discarded blocks return to the transaction pool.

Code:

```
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4
from urllib.parse import urlparse

# Generate a unique id that is in hex
# To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
```

```

self.chain = []
self.transactions = [] # Adding transactions before they are added to a
block
self.create_block(proof = 1, previous_hash = '0')
self.nodes = set() # Set is used as there is no order to be maintained
as the nodes can be from all around the globe

def create_block(self, proof, previous_hash):
    block = {'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions} # Adding transactions to make the blockchain
a cryptocurrency
self.transactions = [] # The list of transaction should become empty
after they are added to a block
self.chain.append(block)
return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False

```

```

previous_proof = previous_block['proof']
proof = block['proof']
hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
if hash_operation[:4] != '0000':
    return False
previous_block = block
block_index += 1
return True

# This method will add the transaction to the list of transactions
def add_transaction(self, sender, receiver, amount):
    self.transactions.append({'sender': sender,
                              'receiver': receiver,
                              'amount': amount})
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1 # It will return the block index to which the
transaction should be added

# This function will add the node containing an address to the set of nodes created in init function
def add_node(self, address):
    parsed_url = urlparse(address) # urlparse will parse the url from the address
    self.nodes.add(parsed_url.netloc) # Add is used and not append as it's a set.
Netloc will only return '127.0.0.1:5000'

# Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the
nodes on the network
def replace_chain(self):
    network = self.nodes # network variable is the set of nodes all around
the globe
    longest_chain = None # It will hold the longest chain when we scan
the network
    max_length = len(self.chain) # This will hold the length of the chain held
by the node that runs this function
    for node in network:
        response = requests.get(f'http://{node}/get_chain') # Use get chain method already
created to get the length of the chain
        if response.status_code == 200:
            length = response.json()['length'] # Extract the length of the chain from
get_chain function
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain): # We check if the length is bigger
and if the chain is valid then
                max_length = length # We update the max length
                longest_chain = chain # We update the longest chain

```

```

        if longest_chain:
            self.chain = longest_chain
        the longest chain
        return True
        return False
# If longest_chain is not none that means it was
# Replace the chain of the current node with
# Return false if current chain is the longest one

```

Part 2 - Mining our Blockchain

Creating a Web App
app = Flask(__name__)

Creating an address for the node on Port 5000. We will create some other nodes as well on different ports

```
node_address = str(uuid4()).replace('-', '')
```

Creating a Blockchain
blockchain = Blockchain()

Mining a new block

```
@app.route('/mine_block', methods = ['GET'])
```

```
def mine_block():
```

```

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) # Hadcoins to
mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block

```

```

    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}

```

```
    return jsonify(response), 200
```

Getting the full Blockchain

```
@app.route('/get_chain', methods = ['GET'])
```

```
def get_chain():
```

```

    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}

```

```
    return jsonify(response), 200
```

```

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the Blockchain
@app.route('/add_transaction', methods = ['POST'])
something to get something in return
def add_transaction():
    json = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes
@app.route('/connect_node', methods = ['POST'])
nodes from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

# Replacing the chain by the longest chain if needed
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():

```

```

is_chain_replaced = blockchain.replace_chain()
if is_chain_replaced:
    response = {'message': 'The nodes had different chains so the chain was replaced by the longest one.',
                'new_chain': blockchain.chain}
else:
    response = {'message': 'All good. The chain is the largest one.',
                'actual_chain': blockchain.chain}
return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5001)

```

Output:

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:5001/add_transaction
- Body Type:** raw
- Request Body (raw):**

```

1  {
2    "amount": 1000,
3    "receiver": "pooja",
4    "sender": "4208036bca004b56b2e456b4daff0059"
5  }
6

```
- Status:** 201 CREATED (3 ms, 226 B)
- Response Body (JSON):**

```

1  {
2    "message": "This transaction will be added to Block 4"
3  }

```

The bottom of the image shows a Windows taskbar with various application icons and a system clock indicating 11:26 on 27-01-2026.

GET http://127.0.0.1:5001/mine_block Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results (1/1) 200 OK 25 ms 621 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "index": 4,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "ed3cc6f96be22e405a6ed26538e3c0a71a79779d682eff6e7e713be9ec690104",
5   "proof": 21391,
6   "timestamp": "2026-01-27 11:26:58.840324",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "4208036bca004b56b2e456b4daff0059"
12    },
13    {
14      "amount": 1000,
15      "receiver": "pooja",
16      "sender": "4208036bca004b56b2e456b4daff0059"
17    }
18  ]
19 }
```

Runner Start Proxy Cookies Vault Trash 11:27 27-01-2026

http://127.0.0.1:5001/get_chain Save Share

GET http://127.0.0.1:5001/get_chain Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 4 ms 533 B

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-01-27 11:09:25.029105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "6d617bfd8543b614960a1afcdbe1e1bb5f00e0e840c15c815761faffe79dec0",
13      "proof": 533,
14      "timestamp": "2026-01-27 11:12:17.003830",
15    }
16  ]
17 }
```

Runner Start Proxy Cookies Vault Trash 11:29 27-01-2026

HTTP client interface showing a POST request to `http://127.0.0.1:5001/connect_node`. The request body is a JSON array of node URLs.

Request:

```
POST http://127.0.0.1:5001/connect_node
```

Body:

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5000",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Response: 201 CREATED (5 ms, 325 B)

Body:

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5000"
6   ]
7 }
```

HTTP client interface showing a GET request to `http://127.0.0.1:5001/replace_chain`. The response is a JSON object containing the actual chain data.

Request:

```
GET http://127.0.0.1:5001/replace_chain
```

Response: 200 OK (3 ms, 1.2 KB)

Body:

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-01-27 11:09:25.029105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "6d617bfd8543b614960a1afcdbe1e1bb5f00e0e840c15c815761faffe79dec0",
13      "proof": 533,
14      "timestamp": "2026-01-27 11:12:17.003830",
15      "transactions": [
16        {
17          "amount": 1,
18          "receiver": "Richard",

```

```

19         "sender": "4208036bca004b56b2e456b4daff0059"
20     }
21 ]
22 },
23 {
24     "index": 3,
25     "previous_hash": "1af95c07c0d568f49621eb2fd8759ff3c729423d818dae19a05b2083fd",
26     "proof": 45293,
27     "timestamp": "2026-01-27 11:19:00.005438",
28     "transactions": [
29         {
30             "amount": 1,
31             "receiver": "Richard",
32             "sender": "4208036bca004b56b2e456b4daff0059"
33         }
34     ],
35     "index": 4,
36     "previous_hash": "ed3cc6f96be22e405a6ed26538e3c0a71a79779d682eff6e7e713be9ec690104",
37     "proof": 21391,
38     "timestamp": "2026-01-27 11:26:58.840324",
39     "transactions": [
40         {
41             "amount": 1,
42             "receiver": "Richard",
43             "sender": "4208036bca004b56b2e456b4daff0059"
44         },
45         {
46             "amount": 1000,
47             "receiver": "pooja",
48             "sender": "4208036bca004b56b2e456b4daff0059"
49         }
50     ],
51     "index": 5,
52     "previous_hash": "1af95c07c0d568f49621eb2fd8759ff3c729423d818dae19a05b2083fd",
53     "proof": 45293,
54     "timestamp": "2026-01-27 11:19:00.005438",
55     "transactions": [
56         {
57             "amount": 1,
58             "receiver": "Richard",
59             "sender": "4208036bca004b56b2e456b4daff0059"
60         }
61     ],
62     "message": "All good. The chain is the largest one."
63 }

```

Conclusion :

Blockchain is a decentralized and secure system for recording transactions. It uses mining, consensus mechanisms, and multinode networks to ensure trust, transparency, and data integrity. Transactions are verified and rewarded through mining, while chain replacement maintains a single valid ledger. Overall, blockchain enables reliable and tamper-proof digital systems without a central authority.